

Document number: P2643R2.

Date: 2024-01-11.

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>.

Authors: Gonzalo Brito Gadeschi, Olivier Giroux, Thomas Rodgers.

Audience: LEWG.

Improving C++ concurrency features

- Improving C++ concurrency features
 - Revisions
 - P2 - (pre-Tokyo submitted)
 - D2 - (post-Varna draft)
 - P1 - (Varna submitted)
 - D1 - (post-Kona draft)
 - Introduction
 - Design
 - Return last observed value on wait success
 - Fallible and timed waiting APIs
 - Predicated waiting APIs
 - Wording
 - Return last observed value from `atomic::wait`
 - Fallible and timed versions of `::wait` APIs
 - Fallible, timed, and predicated versions of `::wait` APIs

Revisions

P2 - (pre-Tokyo submitted)

- Update examples and explanation for Library Evolution audience (forwarded from SG1).
- Rename `try_wait_for` and `try_wait_until` to `wait_for` and `wait_until` for consistency with `condition_variable`.
- Update `wait_with_predicate` to use `condition_variable` semantics.
- `try_wait` is `const noexcept`, `wait_for` / `wait_until` are `const` but may throw exceptions.
- Update list of atomic waiting operations.

- Consistently remove `noexcept` from `wait_for` / `wait_until` -like APIs which can throw due to timeout. Kept `noexcept` on untimed `try_wait` -like APIs.
- Add wording for `latch` APIs.
- Update predicate wait APIs with `_with_predicate` suffix for consistency. TBD whether it can be removed.

D2 - (post-Varna draft)

- Modified the proposal to add a new API instead of modifying the return value of `wait` to avoid breaking the ABI.
- Remove the new C-compatibility free functions except for `atomic_wait_value` and `atomic_flag_try_wait`.
- Add wording for `wait_with_predicate`.

P1 - (Varna submitted)

- Removed timed waiting for freestanding, to reflect Kona guidance, and added discussion.
- Removed pros/cons discussion of returning `optional<T>` vs `pair<bool, T>` vs `T`, reflecting Kona guidance.
- Re-added fallible untimed `try_wait` with rationale, reflecting Kona guidance.

D1 - (post-Kona draft)

- Added discussion of pros/cons of returning `pair<T, bool>` vs. `optional<T>`.
- Added discussion of timed waits and freestanding, given that `<chrono>` is not part of freestanding.
- Added wording for `barrier::try_wait_for` and `barrier::try_wait_until`.
- Added motivating examples.
- Fixed `barrier::try_wait_for/_until` signatures; they were incorrectly accepting an `arrival_token&&`, but since these can be called in a loop consuming the `arrival_token` is incorrect.
- Removed discussion of 'hinted' wait mechanism. The design surface area of this proposal is such that it should be a separate paper, brought forward an interested party.
- Removed fallible untimed `try_wait`.

Introduction

P1135R6 (<https://wg21.link/p1135r6>) introduced several new concurrency primitives to the C++20 concurrency library:

- `<atomic>` : added the class `atomic_flag` , the `wait` and `notify_one/_all` to class template `atomic<>` , and free function versions of these.
- `<semaphore>` : added class template `counting_semaphore<>` and class `binary_semaphore` .
- `<barrier>` , `<latch>` : added class template `barrier<>` and class `latch` .

Though each element included was long coming, and had much implementation experience behind it, fresh user feedback tells us that some improvements could still be made:

1. Return last observed value from `atomic/atomic_ref::wait` ; this value is lost otherwise.
2. Add timed versions of `atomic/atomic_ref/atomic_flag::wait` APIs and other concurrency primitives like `barrier` and `latch` , to make it easier to implement concurrency primitives that expose timed waiting facilities themselves by reusing these (e.g., to enable implementing `<semaphore>` , which already exposes `try_acquire / try_acquire_for / try_acquire_until` , on top of `atomic`).
3. Avoid spurious polling in `atomic/atomic_ref/atomic_flag::wait` by accepting a predicate.

This proposal proposes extensions to address these shortcomings. This branch (<https://gitlab.com/rodgert/gcc/-/commit/dab70764cd68236472656a9394a32185a9ccc890>) demonstrates its implementability in `libstdc++`.

Design

The design of the features above is mostly orthogonal, and this section explores them independently.

Return last observed value on wait success

The design to return the last observed value on wait success adds a new API that returns the old value:

```
template <class T>
T atomic<T>::wait_value(
    T old,
    memory_order order = memory_order::seq_cst
) const noexcept;
```

A new template member is added to respect the WG21 policy of avoiding breaking the ABI of `atomic::wait` .

Example 0: wait-value	
Before	After
<pre>std::atomic<int> a(42); a.wait(42); auto o = a.load(); assert(o != 42); // MAY FAIL!</pre>	<pre>std::atomic<int> a(42); auto o = a.wait_value(42); assert(o != 42); // OK!</pre>

The `atomic<T>::wait_value` method guarantees that the thread is unblocked only if the value changed.

Before this paper, the new `atomic<T>` value that unblocked the wait is not returned to the caller. This has the following two shortcomings:

- **That value is lost forever** (correctness): after the thread is unblocked, the value might change back to the old value before the unblocked thread calls `atomic<T>::load` again (ABA Problem (https://en.wikipedia.org/wiki/ABA_problem)).
- **That value must often be reloaded** (performance): applications need the new value often, which forces them to call `atomic<T>::load` to re-load the value, even though `atomic::wait<T>` had already loaded it (required to test that the value did change preventing spurious unblocking).

After this paper, the value returned by `wait_value` is returned to the caller, eliminating the need for the subsequent load.

API naming

This proposal names this new API `wait_value`. Some other options are:

- `wait_last`
- `wait_fetch`

Fallible and timed waiting APIs

The design of the fallible timed versions of wait APIs adds three new APIs to `atomic`, `atomic_ref`, `atomic_flag`, `barrier`, and `latch` (`semaphore` already has `try_acquire / try_acquire_for`, and `try_acquire_until`). For `atomic` these are

```

template <class T>
optional<T> atomic<T>::try_wait(
    T value,
    memory_order order = memory_order::seq_cst
) const noexcept;

template <class T, class Rep, class Period>
optional<T> atomic<T>::wait_for(
    T value,
    duration<Rep, Period> const& rel_time,
    memory_order order = memory_order::seq_cst
) const;

template <class T, class Clock, class Duration>
optional<T> atomic<T>::wait_until(
    T value,
    time_point<Clock, Duration> const& abs_time,
    memory_order order = memory_order::seq_cst
) const;

```

They are non-blocking, i.e., they eventually return to the caller in a finite-set of steps, even if the value did not change. This enables the application to “do something else” before attempting to wait again.

On failure, i.e., if the value did not change, they return `nullopt` and the operation has no effects (it does not synchronize). On success, they return an `optional<T>` containing the last observed value, which is guaranteed to be different from the one the call site waited on.

The untimed `try_wait` overload waits for a finite *unspecified* duration. The implementation may pick a different duration every time, which is why assigning implementation-specific default arguments to the other untimed wait APIs does not suffice. This overload enables the implementation to attempt to wait for a dynamic system-specific amount of time (e.g. depending on system latencies, load, etc.). Furthermore, `try_wait` is `noexcept`, but the other APIs `wait_for` and `wait_until` may throw timeout-related exceptions.

Since `<chrono>` and `<optional>` are not freestanding, these APIs will not be available in freestanding implementations. C++23+ has mechanisms to partially support these in free-standing. We *should* attempt to support a subset of these new concurrency APIs in freestanding by:

- Supporting the subset of `optional` APIs that do not throw exceptions in freestanding.
- Supporting the `<chrono>` durations in freestanding.

In the following **Example 1**, the atomic variable `t` tracks how many tasks need to be processed. As tasks are processed, this counter is decremented. In the example, the application reports progress by printing the number of remaining tasks every second:

Example 1: Print remaining tasks every 1s.	
Before	After
<pre>std::atomic<int> t; int rem = t.load(); auto b = clock::now(); while (rem != 0) { rem = t.load(); auto e = clock::now() if ((e - b) > 1s) { cout << rem; b = e; } }</pre>	<pre>std::atomic<int> t; int rem = t.load(); while (rem != 0) { auto o = t.wait_for(rem, 1s); rem = o.value_or(rem); cout << rem; }</pre>

Before this proposal, applications need to re-implement `atomic<T>::wait` logic, since it may block for a duration that exceeds the 1s reporting time. Doing this is properly is non-trivial and error prone, e.g., this example accidentally calls `atomic<T>::load` in a loop without any back-off.

After this proposal, the application uses `wait_for` to efficiently and correctly wait for at most 1s.

For `barrier` and `latch`, the proposed fallible wait APIs accept `arrival_token&`, since the token is re-used across multiple API calls. Since C++23, the wait APIs may modify the barrier value and advance the phase, but implementations that do so use `mutable` internally, and this proposal keeps them as `const` methods for consistency with the current wait APIs.

The proposed fallible APIs are the following:

```

template <class CF>
bool barrier<CF>::try_wait(
    arrival_token& tok
) const;
template <class CF, class Rep, class Period>
bool barrier<CF>::wait_for(
    arrival_token& tok,
    duration<Rep, Period> const& rel_time
) const;
template <class CF, class Clock, class Duration>
bool barrier<CF>::wait_until(
    arrival_token& tok,
    time_point<Clock, Duration> const& abs_time
) const;

// bool latch::try_wait() const noexcept; // Available since C++20
template <class Rep, class Period>
bool latch::wait_for(
    duration<Rep, Period> const& rel_time
) const;
template <class Clock, class Duration>
bool latch::wait_until(
    time_point<Clock, Duration> const& abs_time
) const;

```

In the following **Example 2**, an application uses a barrier to track the global amount of tasks to be processed. Once all tasks have been processed, the barrier completes. The processing thread processes its thread-local tasks first, marking the completion of its tasks by arriving at the barrier with the processed task count. Instead of blocking and idling until all tasks have been processed, the processing thread gives other threads 1 ms to complete their tasks, and on failure, it attempts to help other threads by stealing some of their tasks, until all tasks have been completed. In the same way that `arrive` and `wait` enable overlapping independent work in-between arriving and waiting at a barrier, fallible wait methods enable overlapping independent work while waiting on a barrier:

```
// Example 2
std::barrier b(task_count);

// Processing thread:
auto processed_task_count = process_thread_local_tasks();
auto t = b.arrive(processed_task_count);

while (!b.wait_for(t, 1ms)) {
    auto stolen_task_count = steal_and_process_tasks();
    b.arrive(stolen_task_count);
}
```

Predicated waiting APIs

The wait APIs of C++ concurrency primitives wait for a value to change from `x` to some other value. It is very common for applications to need waiting on a more complex condition, e.g., “wait for the value to change to precisely 42”, i.e., “wait until `x == 42`”.

With the current waiting APIs, the application is notified every time the value changes. This is very flexible, since it enables implementing any desired logic on top. The following **Example 3** shows how to wait until `x == 42`:

```
// Example 3: wait until x == 42.
std::atomic<int> x;
int last = x.load();
while (last != 42) {
    // Wait on 'x != last':
    last = x.wait_value(last);
}
assert(last == 42);
```

Unfortunately, this is a forward progress, performance, and energy efficiency “gotcha”. Programs that wait for a condition different from “not equal to” (e.g. “wait for `x == 42`” above) using `atomic::wait` APIs include a re-try loop around the `wait` operation as shown in **Example 3**. The implementation is oblivious to the fact that the program has already been waiting for some time on a more complex condition, and each call to wait in this re-try loop looks to the implementation as the *first* call to wait.

This is problematic, because it leads to re-executing the implementation short-term polling strategy. Implementations do not implement waiting as simple busy-polling (loading the value in a loop). Instead they use concurrent algorithms that depend on “how long has this thread been waiting” to schedule system threads appropriately. If a thread is waiting for the first time, it’ll get many resources to provide low latency in case the condition is met quickly. As the waiting time

increases, threads get less resources, to enable other threads in the system to run. This is crucial for ensuring forward progress of the whole system, since if a waiting thread prevents other threads from running, the condition its waiting on may never be met, causing the application to hang.

A waiting API that accepts a predicate instead of a value enables the application to push the program-defined condition into `atomic::wait`, avoiding the outer re-try loop, and enabling the implementation to track time spent. At least two C++ standard library implementations currently already internally implement `atomic::wait` in terms of a wait taking a predicate.

The proposed design for the predicated `atomic::wait` API is analogous to `condition_variable::wait` (<https://eel.is/c++draft/thread.condition#condvar-13>) API, which take a `stop_waiting` predicate. None of the APIs is `noexcept`, since the predicate is allowed to throw. The design picks an argument order that differs from `condition_variable`: the order of arguments for `condition_variable` is “(lock, chrono duration/time point, predicate)”, but for the proposed APIs, and just like for `atomic::wait_for / _until`, the condition (`old_value` or `stop_predicate`) comes before the chrono types, which comes before the `memory_order` argument which has a default value.

The proposed design for the predicated `atomic::wait` and `atomic_ref::wait` APIs is:

```

// Untimed: blocks.
template <class T, class P>
    requires predicate<P, T>
T atomic<T>::wait_with_predicate(
    P&& stop_waiting,
    memory_order = memory_order::seq_cst
) const;

// Timed, unspecified duration.
template <class T, class P>
    requires predicate<P, T>
optional<T> atomic<T>::try_wait_with_predicate(
    P&& stop_waiting,
    memory_order = memory_order::seq_cst
) const;

// Timed duration
template <class T, class P, class Rep, class Period>
    requires predicate<P, T>
optional<T> atomic<T>::wait_for_with_predicate(
    P&& stop_waiting,
    duration<Rep, Period> const& rel_time,
    memory_order = memory_order::seq_cst
) const;

// Time point
template <class T, class P, class Clock, class Duration>
    requires predicate<P, T>
optional<T> atomic<T>::wait_until_with_predicate(
    P&& stop_waiting,
    time_point<Clock, Duration> const& abs_time,
    memory_order = memory_order::seq_cst
) const;

```

Example 4: before/after vs Example 3.

Before

```

std::atomic<int> x;
int last = x.load();
while (last != 42) {
    // Wait on 'x != last':
    last = x.wait_value(last);
}
assert(last == 42);

```

After

```

std::atomic<int> x;
int last =
    x.wait_with_predicate([](int v) {
        return x == 42;
    });
assert(last == 42);

```

Before this proposal, the application that needs to wait on `x == 42` needs a re-try loop that causes the implementation to pick the short-term polling strategy every time `x` changes.

After this proposal, the application passes a predicate to wait on `x == 42`. While `x` may change many times until this predicate is satisfied, the implementation is aware that `x` changing is not the condition the application is waiting on.

Wording

Return last observed value from `atomic::wait`

Add new APIs to the list of atomic waiting operations in the Note at [atomics.wait#2]

(<https://eel.is/c++draft/atomics.wait#2>):

[Note 2: The following functions are atomic waiting operations:

1. `atomic<T>::wait` and `atomic<T>::wait_value`,
 2. `atomic_flag::wait`,
 3. `atomic_wait` ~~and~~, `atomic_wait_explicit`, `atomic_wait_value`, and `atomic_wait_value_explicit`,
 4. `atomic_flag_wait` and `atomic_flag_wait_explicit`, and
 5. `atomic_ref<T>::wait` and `atomic_ref<T>::wait_value`.
- end note]

To [atomics.syn] (<http://eel.is/c++draft/atomics.syn>):

```

namespace std {
    // [atomics.nonmembers], non-member functions
    template<class T>
    void atomic_wait(const volatile atomic<T>*,
                    typename atomic<T>::value_type) noexcept; //
    template<class T>
    void atomic_wait(const atomic<T>*, typename atomic<T>::value_type) noexcept; //
    template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*,
                             typename atomic<T>::value_type,
                             memory_order) noexcept; //
    template<class T>
    void atomic_wait_explicit(const atomic<T>*, typename atomic<T>::value_type, //
                             memory_order) noexcept;

    template<class T>
    typename atomic<T>::value_type
    atomic_wait_value(const volatile atomic<T>*, // freestanding
    typename atomic<T>::value_type) noexcept;
    template<class T>
    typename atomic<T>::value_type
    atomic_wait_value(const atomic<T>*, // freestanding
    typename atomic<T>::value_type) noexcept;
    template<class T>
    typename atomic<T>::value_type
    atomic_wait_value_explicit(const volatile atomic<T>*, // freestanding
    typename atomic<T>::value_type,
    memory_order) noexcept;
    template<class T>
    typename atomic<T>::value_type
    atomic_wait_value_explicit(const atomic<T>*, // freestanding
    typename atomic<T>::value_type,
    memory_order) noexcept;
}

```

To [atomics.ref.generic.general] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        T wait_value(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```
void wait(T old, memory_order order = memory_order::seq_cst) const noexcept;  
T wait value(T old, memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions:* order is neither memory_order::release nor memory_order::acq_rel.
- *Effects:* Repeatedly performs the following steps, in order:
 1. Evaluates load(order) and compares its value representation for equality against that of old.
 2. If they compare unequal, wait returns and wait_value returns the result of the evaluation of load(order) in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks:* This function is an atomic waiting operation (atomics.wait (<http://eel.is/c++draft/atomics#wait>)) on atomic object *ptr.

To [atomics.ref.int] (<http://eel.is/c++draft/atomics.ref.int>):

```
namespace std {  
    template<> struct atomic_ref<integral> {  
        integral wait value(integral, memory_order = memory_order::seq_cst) const noex  
    };  
}
```



To [atomics.ref.float] (<http://eel.is/c++draft/atomics.ref.float>):

```
namespace std {  
    template<> struct atomic_ref<floating-point> {  
        floating-point wait value(floating-point, memory_order = memory_order::seq_cst  
    };  
}
```



To [atomics.ref.pointer] (<http://eel.is/c++draft/atomics.ref.pointer>):

```
namespace std {  
    template<class T> struct atomic_ref<T*> {  
        T* wait value(T*, memory_order = memory_order::seq_cst) const noexcept;  
    };  
}
```

To [atomics.types.generic.general] (<http://eel.is/c++draft/atomics.types.generic.general>):

```

namespace std {
    template<class T> struct atomic {
        T wait value(T, memory_order = memory_order::seq_cst) const volatile noexcept;
        T wait value(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```

void wait(T old, memory_order order = memory_order::seq_cst) const volatile noexce
void wait(T old, memory_order order = memory_order::seq_cst) const noexcept;
T wait value(T old, memory_order order = memory_order::seq_cst) const volatile noe
T wait value(T old, memory_order order = memory_order::seq_cst) const noexcept;

```

- *Preconditions*: order is neither memory_order::release nor memory_order::acq_rel .
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates load(order) and compares its value representation for equality against that of old .
 2. If they compare unequal, wait returns and wait_value returns the result of the evaluation of load(order) in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: ~~This function is an~~These functions are atomic waiting operations (atomics.wait (<http://eel.is/c++draft/atomics#wait>)).

To [atomics.types.int] (<http://eel.is/c++draft/atomics.types.int>):

```

namespace std {
    template<> struct atomic<integral> {
        integral wait value(integral, memory_order = memory_order::seq_cst) const vola
        integral wait value(integral, memory_order = memory_order::seq_cst) const noex
    };
}

```

To [atomics.types.float] (<http://eel.is/c++draft/atomics.types.float>):

```

namespace std {
    template<> struct atomic<floating-point> {
        floating-point wait value(floating-point, memory_order = memory_order::seq_cst
        floating-point wait value(floating-point, memory_order = memory_order::seq_cst
    };
}

```

To [atomics.types.pointer] (<http://eel.is/c++draft/atomics.types.pointer>):

```

namespace std {
    template<class T> struct atomic<T*> {
        T* wait value(T*, memory_order = memory_order::seq_cst) const volatile noexcept
        T* wait value(T*, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [util.smartptr.atomic.shared] (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```

namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        shared_ptr<T> wait value(shared_ptr<T> old, memory_order = memory_order::seq_c
    };
}

```

and

```

void wait_value(shared_ptr<T> old, memory_order order = memory_order::seq_cst) con
shared_ptr<T> wait value(shared_ptr<T> old, memory_order order = memory_order::se

```

- *Preconditions*: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, `wait` returns and `wait_value` returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: Two `shared_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. ~~This function is an~~ These functions are atomic waiting operations (`atomics.wait` (<http://eel.is/c++draft/atomics.wait>)).

To `[util.smartptr.atomic.weak]` (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        weak_ptr<T> wait value(weak_ptr<T> old, memory_order = memory_order::seq_cst)
    };
}
```

```
void wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst) const noexcept
weak_ptr<T> wait value(weak_ptr<T> old, memory_order order = memory_order::seq_cst)
```

- *Preconditions:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects:* Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, `wait` returns and `wait_value` returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks:* Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. ~~This function is an~~ These functions are atomic waiting operations (`atomic.wait` (<http://eel.is/c++draft/atomic.wait>)).

No changes to `[atomic.nonmembers]` (<http://eel.is/c++draft/atomic.nonmembers>) are needed.

No changes to `[atomic.flag]` (<http://eel.is/c++draft/atomic.flag>)'S `wait` APIs are needed.

Fallible and timed versions of `::wait` APIs

Add new APIs to the list of atomic waiting operations in the Note at `[atomic.wait#2]`

(<https://eel.is/c++draft/atomic.wait#2>):

[Note 2: The following functions are atomic waiting operations:

1. `atomic<T>::wait`, `atomic<T>::try_wait`, `atomic<T>::wait_for`, `atomic::<T>::wait_until`,
2. `atomic_flag::wait`, `atomic_flag::try_wait`, `atomic_flag::wait_for`,
`atomic_flag::wait_until`,
3. `atomic_wait` ~~and~~, `atomic_wait_explicit`, `atomic_try_wait`, and
`atomic_try_wait_explicit`,

4. atomic_flag_wait ~~and~~, atomic_flag_wait_explicit, atomic_flag_try_wait,
atomic_flag_try_wait_explicit, ~~and~~
 5. atomic_ref<T>::wait, atomic_ref<T>::try_wait, atomic_ref<T>::wait_for,
atomic_ref<T>::wait_until.
- end note]

To [atomics.syn] (<http://eel.is/c++draft/atomics.syn>):

EDITORIAL: only APIs that do not use <optional> or <chrono> added for C compatibility. That is, only try_wait is added for C compatibility, wait_for and wait_until are not added here.

```
namespace std {
    // [atomics.flag], flag type and operations

    void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept; // freestandi
    void atomic_flag_wait(const atomic_flag*, bool) noexcept;           // freestandi
    void atomic_flag_wait_explicit(const volatile atomic_flag*,         // freestandi
                                   bool, memory_order) noexcept;
    void atomic_flag_wait_explicit(const atomic_flag*,                  // freestandi
                                   bool, memory_order) noexcept;

    bool atomic_flag_try_wait(const volatile atomic_flag*, bool) noexcept; // freest
    bool atomic_flag_try_wait(const atomic_flag*, bool) noexcept;           // freest
    bool atomic_flag_try_wait_explicit(const volatile atomic_flag*,         // freest
                                     bool, memory_order) noexcept;
    bool atomic_flag_try_wait_explicit(const atomic_flag*,                  // freest
                                     bool, memory_order) noexcept;
}
```

To [atomics.ref.generic.general] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        optional<T> try_wait(T, memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T> wait for(
            T, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<T> wait until(
            T, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const;
    };
}

```

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```

optional<T> try_wait(T old, memory_order = memory_order::seq_cst) const noexcept;
template <class Rep, class Period>
optional<T> wait for(T old,
    chrono::duration<Rep, Period> const& rel time,
    memory_order order = memory_order::seq_cst
) const;
template <class Clock, class Duration>
optional<T> wait until(T old,
    chrono::time_point<Clock, Duration> const& abs time,
    memory_order order = memory_order::seq_cst
) const;

```

- Preconditions: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs time` (for `wait until`) or when at least `rel time` has passed from

the start of the function (for wait_for).

The timeout for try_wait is finite but otherwise unspecified.

- **INCONSISTENCY:** try_wait is noexcept, but here we say it throws. Add above "...and it does not throw.". Add below: "...from wait_for and wait_until."
- Throws: Timeout-related exceptions (thread.req.timing_(<http://eel.is/c++draft/thread.req.timing>)).
- Remarks: This function is an atomic waiting operation (atomics.wait_(<http://eel.is/c++draft/atomics#wait>)) on atomic object *ptr.

To [atomics.ref.int] (<http://eel.is/c++draft/atomics.ref.int>):

```
namespace std {  
    template<> struct atomic_ref<integral> {  
        optional<integral> try_wait(integral, memory_order = memory_order::seq_cst) co  
        template <class Rep, class Period>  
        optional<integral> wait_for(  
            integral, chrono::duration<Rep, Period> const& rel time,  
            memory_order = memory_order::seq_cst  
        ) const;  
        template <class Clock, class Duration>  
        optional<integral> wait_until(  
            integral, chrono::time_point<Clock, Duration> const& abs time,  
            memory_order = memory_order::seq_cst  
        ) const;  
    };  
}
```

To [atomics.ref.float] (<http://eel.is/c++draft/atomics.ref.float>):

```

namespace std {
    template<> struct atomic_ref<floating-point> {
        optional<floating-point> try_wait(
            floating-point,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<floating-point> wait_for(
            floating-point, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<floating-point> wait_until(
            floating-point, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const;
    };
}

```

To [atomics.ref.pointer] (<http://eel.is/c++draft/atomics.ref.pointer>):

```

namespace std {
    template<class T> struct atomic_ref<T*> {
        optional<T*> try_wait(T* old, memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T*> wait_for(
            T*, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<T*> wait_until(
            T*, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const;
    };
}

```

To [atomics.types.generic.general] (<http://eel.is/c++draft/atomics.types.generic.general>):

```

namespace std {
    template<class T> struct atomic {
        optional<T> try_wait(T old, memory_order = memory_order::seq_cst) const noexcept;
        optional<T> try_wait(T old, memory_order = memory_order::seq_cst) const volatile;
        template <class Rep, class Period>
        optional<T> wait_for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const;
        optional<T> wait_for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile;
        template <class Clock, class Duration>
        optional<T> wait_until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<T> wait_until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```

optional<T> try_wait(T old, memory_order = memory_order::seq_cst) const noexcept;
optional<T> try_wait(T old, memory_order = memory_order::seq_cst) const volatile;
template <class Rep, class Period>
optional<T> wait_for(T old,
                    chrono::duration<Rep, Period> const& rel_time,
                    memory_order order = memory_order::seq_cst
                    ) const;
template <class Rep, class Period>
optional<T> wait_for(T old,
                    chrono::duration<Rep, Period> const& rel_time,
                    memory_order order = memory_order::seq_cst
                    ) const volatile;
template <class Clock, class Duration>
optional<T> wait_until(T old,
                     chrono::time_point<Clock, Duration> const& abs_time,
                     memory_order order = memory_order::seq_cst
                     ) const;
template <class Clock, class Duration>
optional<T> wait_until(T old,
                     chrono::time_point<Clock, Duration> const& abs_time,
                     memory_order order = memory_order::seq_cst
                     ) const volatile;

```

- Preconditions: `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `wait_until`) or when at least `rel_time` has passed from the start of the function (for `wait_for`).

The timeout for `try_wait` is finite but otherwise unspecified.

- **INCONSISTENCY**: `try_wait` is `noexcept`, but here we say it throws. Add above "...and it does not throw.". Add below: "...from `wait_for` and `wait_until`."
- Throws: Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).

- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To `[atomics.types.int]` (<http://eel.is/c++draft/atomics.types.int>):

```
namespace std {
    template<> struct atomic<integral> {
        optional<integral> try wait(integral, memory_order = memory_order::seq_cst) co
        optional<integral> try wait(integral, memory_order = memory_order::seq_cst) co
        template <class Rep, class Period>
        optional<integral> wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Rep, class Period>
        optional<integral> wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile;
        template <class Clock, class Duration>
        optional<integral> wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<integral> wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile;
    };
}
```

To `[atomics.types.float]` (<http://eel.is/c++draft/atomics.types.float>):

```

namespace std {
    template<> struct atomic<floating-point> {
        optional<floating-point> try wait(floating-point, memory_order = memory_order::
optional<floating-point> try wait(floating-point, memory_order = memory_order:
        template <class Rep, class Period>
        optional<floating-point> wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Rep, class Period>
        optional<floating-point> wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile;
        template <class Clock, class Duration>
        optional<floating-point> wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<floating-point> wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile;
    };
}

```

To [atomics.types.pointer] (<http://eel.is/c++draft/atomics.types.pointer>):

```

namespace std {
    template<class T> struct atomic<T*> {
        optional<T*> try_wait(T*, memory_order = memory_order::seq_cst) const noexcept
        optional<T*> try_wait(T*, memory_order = memory_order::seq_cst) const volatile
        template <class Rep, class Period>
        optional<T*> wait_for(
            T*, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Rep, class Period>
        optional<T*> wait_for(
            T*, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const volatile;
        template <class Clock, class Duration>
        optional<T*> wait_until(
            T*, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<T*> wait_until(
            T*, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const volatile;
    };
}

```

To [util.smartptr.atomic.shared] (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```

optional<shared_ptr<T>> try_wait(
    shared_ptr<T> old,
    memory_order order = memory_order::seq_cst
) const noexcept;
template <class Rep, class Period>
optional<shared_ptr<T>> wait_for(
    shared_ptr<T> old,
    chrono::duration<Rep, Period> const& rel_time,
    memory_order order = memory_order::seq_cst
) const;
template <class Clock, class Duration>
optional<shared_ptr<T>> wait_until(
    shared_ptr<T> old,
    chrono::time_point<Clock, Duration> const& abs_time,
    memory_order order = memory_order::seq_cst
) const;

```

- Preconditions: `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `wait_until`) or when at least `rel_time` has passed from the start of the function (for `wait_for`).

The timeout for `try_wait` is finite but otherwise unspecified.

- **INCONSISTENCY**: `try_wait` is `noexcept`, but here we say it throws. Add above "...and it does not throw.". Add below: "...from `wait_for` and `wait_until`."
- Throws: Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- Remarks: Two `shared_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. These functions are atomic waiting operations (`atomics.wait` (<http://eel.is/c++draft/atomics.wait>)).

To [`util.smartptr.atomic.weak`] (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        optional<weak_ptr<T>> try_wait(
            weak_ptr<T>,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<weak_ptr<T>> wait_for(
            weak_ptr<T>l, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        optional<weak_ptr<T>> wait_until(
            weak_ptr<T>, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const;
    };
}
```

```

optional<weak_ptr<T>> try_wait(
    weak_ptr<T> old,
    memory_order order = memory_order::seq_cst
) const noexcept;
template <class Rep, class Period>
optional<weak_ptr<T>> wait_for(
    weak_ptr<T> old,
    chrono::duration<Rep, Period> const& rel_time,
    memory_order order = memory_order::seq_cst
) const;
template <class Clock, class Duration>
optional<weak_ptr<T>> wait_until(
    weak_ptr<T> old,
    chrono::time_point<Clock, Duration> const& abs_time,
    memory_order order = memory_order::seq_cst
) const;

```

- Preconditions: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `wait_until`) or when at least `rel_time` has passed from the start of the function (for `wait_for`).

The timeout for `try_wait` is finite but otherwise unspecified.

- **INCONSISTENCY**: `try_wait` is `noexcept`, but here we say it throws. Add above "...and it does not throw.". Add below: "...from `wait_for` and `wait_until`."
- Throws: Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- Remarks: Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. These functions are atomic waiting operations (`atomics.wait` (<http://eel.is/c++draft/atomics.wait>)).

To [`atomic.flag`] (<http://eel.is/c++draft/atomics.flag>):

```

namespace std {
    struct atomic_flag {
        bool try_wait(bool, memory_order = memory_order::seq_cst) const noexcept;
        bool try_wait(bool, memory_order = memory_order::seq_cst) const volatile noexcept;
        template <class Rep, class Period>
        bool wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Rep, class Period>
        bool wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const volatile;
        template <class Clock, class Duration>
        bool wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const;
        template <class Clock, class Duration>
        bool wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const volatile;
    };
}

```

```

bool atomic_flag::try_wait(const atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait(const volatile atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait_explicit(const atomic_flag* object, bool old, memory_order order) const;
bool atomic_flag::try_wait_explicit(const volatile atomic_flag* object, bool old, memory_order order) const volatile;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) const;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) const volatile;

```

For `atomic_flag::try_wait` let `order` be `memory_order::seq_cst`. Let `flag` be `object` for the non-member functions, and `this` for the member functions.

- *Preconditions:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects:* Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.

3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs time` (for `wait until`) or when at least `rel time` has passed from the start of the function (for `wait for`).

The timeout for `try wait` is finite but otherwise unspecified.

- *Throws:* Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To `[thread.barrier]` (<http://eel.is/c++draft/thread.barrier>):

```
namespace std {  
    template <class Completion Function>  
    class barrier {  
  
    public:  
        bool try wait(arrival token& tok) const;  
        template <class Rep, class Period>  
        bool wait for(arrival token& tok, chrono::duration<Rep, Period> const& rel tim  
        template <class Clock, class Duration>  
        bool wait until(arrival token& tok, chrono::time point<Clock, Duration> const&  
    };  
}
```

```
bool try wait(arrival token& tok) const;  
template <class Rep, class Period>  
bool wait for(arrival token& tok, chrono::duration<Rep, Period> const& rel time) c  
template <class Clock, class Duration>  
bool wait until(arrival token& tok, chrono::time point<Clock, Duration> const& abs
```

- *Preconditions:* `arrival` is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.
- *Effects:* Blocks at the synchronization point associated with `arrival` until the phase completion step of the synchronization point's phase is run or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `false`; otherwise, it returns `true`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `wait_until`) or when at least `rel_time` has passed from the start of the function (for `wait_for`).

The timeout for `try_wait` is finite but otherwise unspecified.

An implementation must ensure that `wait_for` and `wait_until` do not consistently return `false` after the phase completion step associated with `arrival` has run.

[Note: If `arrival` is associated with the synchronization point for a previous phase, the call returns immediately. — end note]

- **Throws:** `system_error` when an exception is required (`thread.req.exception` (<http://eel.is/c++draft/thread.req.exception>)) or timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- **Error conditions:** Error conditions: Any of the error conditions allowed for mutex types (`thread.mutex.requirements.mutex` (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).

To `thread.latch` (<http://eel.is/c++draft/thread.latch>):

```
namespace std {  
    class latch {  
    public:  
        bool try_wait() const noexcept;  
        template <class Rep, class Period>  
        bool wait_for(chrono::duration<Rep, Period> const& rel_time) const;  
        template <class Clock, class Duration>  
        bool wait_until(chrono::time_point<Clock, Duration> const& abs_time) const;  
    };  
}
```

```
bool try_wait() const noexcept;
```

- **Returns:** ~~With very low probability `false`. Otherwise `counter == 0`.~~

SG1: the change below reformulates `try_wait` in terms of a timeout. This seems equivalent to the current formulation, but may be a breaking change.

```
template <class Rep, class Period>  
bool wait_for(chrono::duration<Rep, Period> const& rel_time) const;  
template <class Clock, class Duration>
```

bool wait_until(chrono::time_point<Clock, Duration> const& abs_time) const;

- Effects: If counter equals zero, return immediately. Otherwise, blocks on `*this` until a call to `count_down` that decrements counter to zero or the timeout expires. If it is unblocked by the timeout there is no effect and it returns `false`; otherwise, it returns `true`.

The timeout expires (thread.req.timing (http://eel.is/c++draft/thread.req.timing)) when the current time is after `abs_time` (for `wait_until`) or when at least `rel_time` has passed from the start of the function (for `wait_for`).

The timeout for `try_wait` is finite but otherwise unspecified.

- Throws: `system_error` when an exception is required (thread.req.exception (http://eel.is/c++draft/thread.req.exception)) or timeout-related exceptions (thread.req.timing (http://eel.is/c++draft/thread.req.timing)).
- Error conditions: Error conditions: Any of the error conditions allowed for mutex types (thread.mutex.requirements.mutex (http://eel.is/c++draft/thread.mutex.requirements.mutex)).

Fallible, timed, and predicated versions of `::wait` APIs

Add new APIs to the list of atomic waiting operations in the Note at [atomics.wait#2]

(https://eel.is/c++draft/atomics.wait#2):

[Note 2: The following functions are atomic waiting operations:

1. `atomic<T>::wait`, `atomic<T>::wait_with_predicate`,
`atomic<T>::try_wait_with_predicate`, `atomic<T>::wait_for_with_predicate`,
`atomic::<T>::wait_until_with_predicate`,
 2. `atomic_flag::wait`, `atomic_flag::wait_with_predicate`, `try_wait_with_predicate`,
`wait_for_with_predicate`, `wait_until_with_predicate`,
 3. `atomic_wait` and `atomic_wait_explicit`,
 4. `atomic_flag_wait` and, `atomic_flag_wait_explicit`, and
 5. `atomic_ref<T>::wait`, `atomic_ref<T>::wait_with_predicate`,
`atomic_ref<T>::try_wait_with_predicate`,
`atomic_ref<T>::wait_for_with_predicate`,
`atomic_ref<T>::wait_until_with_predicate`.
- end note]

To [atomics.ref.generic.general] (http://eel.is/c++draft/atomics.ref.generic.general):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        template <class T, class P>
        requires predicate<P, T>
        T wait with predicate(
            P&& stop waiting,
            memory_order = memory_order::seq_cst) const;
        template <class T, class P>
        requires predicate<P, T>
        optional<T>
        try wait with predicate(
            P&& stop waiting,
            memory_order = memory_order::seq_cst) const;
        template <class T, class P, class Rep, class Period>
        requires predicate<P, T>
        optional<T> wait for with predicate(
            duration<Rep, Period> const& rel time,
            P&& stop waiting,
            memory_order = memory_order::seq_cst) const;
        template <class T, class P, class Clock, class Duration>
        requires predicate<P, T>
        optional<T> wait until with predicate(
            P&& stop waiting,
            time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst) const;
    };
}

```

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```

template <class T, class P>
    requires predicate<P, T>
T wait with predicate(
    P&& stop waiting,
    memory_order = memory_order::seq_cst) const;
template <class T, class Predicate>
    requires predicate<P, T>
optional<T> try wait with predicate(
    P&& stop waiting,
    memory_order = memory_order::seq_cst) const;
template <class T, class Rep, class Period, class Predicate>
    requires predicate<P, T>
optional<T> wait for with predicate(
    duration<Rep, Period> const& rel_time,
    P&& stop waiting,
    memory_order = memory_order::seq_cst) const;
template <class T, class Clock, class Duration, class Predicate>
    requires predicate<P, T>
optional<T> wait until with predicate(
    P&& stop waiting,
    time_point<Clock, Duration> const& abs_time,
    memory_order = memory_order::seq_cst) const;

```

- Preconditions: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and calls `stop_predicate` with its result.
 2. If `stop_predicate` returns `true`, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation, or is unblocked spuriously, or if the timeout of `try_wait_with_predicate`, `wait_for_with_predicate`, or `wait_until_with_predicate` expired. If `try_wait_with_predicate`, `wait_for_with_predicate`, or `wait_until_with_predicate` are unblocked by the timeout there is no effect and these return `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `wait_until_with_predicate`) or when at least `rel_time` has passed from the start of the function (for `wait_for_with_predicate`).

The timeout for `try_wait_with_predicate` is finite but otherwise unspecified.

- Throws: any exception thrown by `stop waiting`. The `wait for with predicate` and `wait until with predicate` also throw timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).

- *Remarks:* These functions are atomic waiting operations (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

EDITORIAL: intentionally omitting all other modifications required for the predicated APIs until initial design feedback from LEWG. But intended to be analogous for all `atomic_ref` and `atomic_specializations`, and for `atomic_flag`.